

Unit Testing (JUnit y Mockito)

¿Qué es Unit Testing y qué problema resuelve?

Un unit test es una prueba automatizada donde ponemos a prueba un bloque de código muy específico (como una clase o un método) para validar que responda como esperamos, con esto no dependemos de servicios externos como la bd o APIs. El testing resuelve el problema de la regresión que es que garantiza que el código que funcionaba ayer siga funcionando hoy, incluso al agregar nuevas funcionalidades o refactorizar. Como esta totalmente aislados, estos tests se ejecutan en milisegundos. La ventaja en el flujo de trabajo es que, si algo falla después de un cambio, el test nos señala exactamente el método y la línea donde ocurrió el problema, y nos deja corregirlo antes de subir el código.

¿Dónde se ve en mi código?

Implementé las pruebas a lo largo de 4 proyectos del repositorio:

Aserciones y assertThrows (01-junit-fundamentos): En BoletaTest.java usé assertTrue y assertFalse para probar las fronteras exactas de calificación (69, 70, 71). Para el caso de error, en 02-junit-catalogo/src/.../ExcepcionesTest.java, utilicé assertThrows para verificar que, al intentar inscribir a un alumno sin cupo disponible, el sistema lance CupoLlenoException y valide que el mensaje de error tenga la clave del curso.

Ciclo de vida (01-junit-fundamentos): En CicloDeVidaTest.java apliqué @BeforeEach para instanciar una boleta limpia antes de cada prueba y @AfterEach para limpieza de recursos. Esto sirvió para entender en la práctica que JUnit instancia la clase de test en cada prueba para que los tests sean independientes entre sí.

Mockito (@Mock, @InjectMocks, when/verify):

En 04-mockito-dobles/src/.../ServicioInscripcionTest.java fue poner a prueba un servicio real que interactúa con bases de datos y notificaciones.

- @Mock para simular el RepositorioAlumnos y el Notificador.
- @InjectMocks sobre ServicioInscripcion para que Mockito inyecte automáticamente esos objetos falsos en el servicio.
- when(repoAlumnos.buscar("A01")).thenReturn(...) programa el comportamiento para que el mock devuelva un alumno específico.
- verify(notificador).enviarRechazo(...) comprueba que el servicio llamara al sistema de correos. También verify(notificador, never()).enviarConfirmacion(...) para garantizar que no se le enviara un correo de éxito a alguien rechazado.

¿Por qué mockear ahí y qué colaborador no mockearía? RepositorioAlumnos porque una base de datos real hace que los tests sean lentos (300 ms por consulta). También Notificador porque tiene un efecto externo: si se usa el real, mandaría correos de verdad cada vez que corro la suite, lo cual es no debe hacerse en un entorno de pruebas.

¿Qué no mockearía? Las clases de dominio que contienen reglas de negocio. En este proyecto, la clase Curso tiene la regla de cuántos lugares hay disponibles. Si le pongo un @Mock a Curso, dejo de probar su comportamiento real y el test aprobaría lo que esta proframado, dándome un falso verde. Todo lo que sea un record, entidad u objeto de valor se debe instanciar con new.

Qué pasa si no lo uso (El experimento ver-fallar.sh)

Sin testing, dependes de revisiones manuales, y los bugs se van a producción. Para comprobar que una suite en verde no siempre significa "código a salvo", corrimos el experimento ver-fallar.sh en el proyecto 01. Este script inyectó un bug cambiando la regla de aprobación de ≥ 70 a > 70 (cambiar un solo carácter). El resultado: 20 de los 22 tests siguieron pasando en verde. Los tests que probaban "sacó 90" o "sacó 50" no detectaron nada porque estaban muy lejos de la frontera de decisión. Solo 2 tests fallaron, los que probaban el límite crítico (sacó 70). Con esto aprendí que los tests no sirven si no atacan las fronteras exactas del código que serían los boundary values.

Cómo correrlo

Desde la terminal, para ejecutar las pruebas con Mockito en el proyecto 04:

Acceder al directorio: `cd 04-mockito-dobles`

Ejecutar la suite: `./mvnw test` (Para el experimento del bug, ejecutar `./scripts/ver-fallar.sh` en la carpeta 01-junit-fundamentos).